

第 1 章 数制系统与转换

学习目标

1. 简介

本章的第一部分介绍了本书后面将要学到的内容。你不仅要能够从总体上掌握这部分内容,还应该能够解释以下内容:

- (1) 模拟系统与数字系统的区别,以及为什么数字系统有更高的精确度。
- (2) 组合电路与时序电路的区别。
- (3) 为什么在数字系统中广泛采用二值信号及二进制数字。

2. 数制系统与转换

当学习完本章,你应该可以解决下列几类问题:

- (1) 给定一个某基数(从 2~16)下正的纯整数、纯小数或者混合数,可以将其转换成其他基数下的值,并将该数写成幂数展开式来证明你的处理过程是正确的。
- (2) 正二进制数的加法、减法、乘法和除法运算,并根据进位和借位来解释加法与减法运算的过程。
- (3) 会用原码、反码和补码形式表示负数,会用反码和补码完成带符号二进制数的加法运算,并证明自己所使用的方法,能说明什么情况下会发生溢出。
- (4) 可以用 BCD 码(binary-coded-decimal)、6-3-1-1 码及余 3 码(excess-3 code)等编码表示十进制数。当给定一组权值时可以构造出相应的有权码。

习指导

1. 学习第 1.1 节数字系统与开关电路,并回答下列问题

- (a) 模拟系统与数字系统的基本区别是什么?
- (b) 为什么数字系统比模拟系统有更高的精确度?
- (c) 解释组合开关电路与时序开关电路的区别。
- (d) 数字系统中的开关器件有哪些共同特性?
- (e) 为什么数字系统采用二进制数?

2. 学习第 1.2 节数制系统与转换,在学习过程中回答下列问题

- (a) 当使用除法方法完成不同基下的数制转换时,最先得到的余数是最低数字位还是最高数字位?
- (b) 完成本教材中的所有例题,确保你理解其中的每一个步骤。

(c) 式(1-1)介绍了一种实现二进制数与十六进制数相互转换的简易方法。你为什么是从二进制小数点处而不是从最左边开始构造 4 位分组?

(d) 为什么不可能像十六进制一样将十进制数逐个数字转换成二进制?

(e) 完成下面的转换表。

二进制 (以 2 为基)	八进制 (以 8 为基)	十进制 (以 10 为基)	十六进制 (以 16 为基)
0	0	0	0
1			
10			
11			
100			
101			
110			
111			
1000			
1001			
1010			
1011			
1100			
1101			
1110			
1111			
10000	20	16	10

(f) 完成习题 1.1~习题 1.4。

3. 学习第 1.3 节二进制运算

(a) 确保你可以理解所有例子,尤其是减法过程中的借位传播过程。

(b) 为了确保理解借位处理过程,请写出下列例子以 2 的幂次进行运算时的详细分析过程。

$$\begin{array}{r}
 1100 \\
 -101 \\
 \hline
 111
 \end{array}$$

4. 完成习题 1.5~习题 1.17(a)

5. 学习第 1.4 节负数的表示

(a) 在数字系统中,为什么通常采用反码和补码来表示负数,而不采用原码表示?

(b) 说出两种不同的求 n 位二进制数反码形式的方法。

(c) 说出 3 种不同的求 n 位二进制数补码形式的方法。

(d) 若字长 $n=4$ 位(包含符号位),原码数 1000_2 表示哪个十进制数? 若它是补码,表示什么数? 若它是反码,表示什么数?

(e) 给出某个负数的补码形式,你如何求出它的真值? 给出某个负数的反码形式,你如何求出它的真值?

(f) 若字长 $n=6$ 位(包含符号位),原码数 100000_2 表示哪个十进制数? 若它是补码,表

示什么数? 若它是反码, 表示什么数?

(g) 溢出的含义是什么? 在进行补码加法或反码加法运算中, 如何判断是否发生溢出?

最后一位有进位是否意味着发生了溢出?

(h) 举例并完成正数与负数在不同组合下的反码与补码加法运算。

(i) 如何证明反码加法中使用的循环进位方法的正确性?

(j) 补码数中经常引起麻烦的一种情况是负数的一个特例, 此时该补码数由一个 1 和其后的所有 0 组成(1000...000)。若这个数字的长度为 n 位, 它代表什么数? 为什么? (它并不是负数零。)

(k) 完成习题 1.7 和习题 1.8。

6. 学习第 1.5 节二进制编码

(a) 分别用 BCD 码、余 3 码、6-3-1-1 码和五中取二码(2-out-of-5 code)表示数字 187。

(b) 验证 6-3-1-1 编码是一种有权码。注意该编码方案中有些十进制数字有两种不同的编码组合, 例如 0101 和 0110 都可以表示 4, 在这种情况下, 我们采用具有较小二进制值的编码组合。

(c) 如何求取余 3 码?

(d) 如何求取十进制数字的 ASCII 编码? ASCII 编码中大写字母与小写字母的代码之间有何关系?

(e) 完成习题 1.9。

7. 测验前的准备

如果把本课程作为自学课程, 那么在继续学习下一章之前, 你需要先通过为本章准备的测验。这个测验的目的是检测你是否已经掌握了本章的所有内容, 以及是否已经做好学习下一章的准备。在进行这个测验之前你需要:

(a) 将你的答案与本书最后提供的答案作对照。如果某些答案存在错误, 确保你明白为什么出现错误并能够改正这些错误。

(b) 确保你已经实现本章一开始所列出的所有学习目标。

1.1 数字系统与开关电路

数字系统广泛应用于计算机、数据处理、控制系统、通信及测量等领域。由于数字系统比模拟系统具有更高的精确度和可靠性, 因此以前用模拟系统完成的许多任务现在都采用数字化来实现。

在数字系统中, 物理量或信号都被认为是离散值, 但在模拟系统中, 物理量或信号可以在某个给定的范围内连续变化。例如, 数字系统中的输出电压被限定为 0V 或 5V 两个值, 但模拟系统中的输出电压允许取值为 $-10\text{V} \sim +10\text{V}$ 之间的任何值。

因为数字系统采用离散量工作, 在许多情况下, 这些系统被设计成在给定的输入下输出结果恰好是正确的。例如, 若我们用十进制乘法器将两个 5 位的十进制数相乘, 结果中的 10 位十进制乘积全部是正确的。但另一方面, 模拟乘法器的输出结果却可能存在一个到数个百分点的误差范围, 这取决于构建该模拟乘法器的部件的精度。此外, 如果我们要求乘积

结果为 20 位而不是 10 位时所得到的结果也必须是正确的,这时我们可以重新设计数字乘法器,以便可以处理更多的数字位数,可以对输入端输入更多的位数。但由于受到部件精度的限制,对模拟乘法器在精度方面做类似的改进几乎是不可能的。

数字系统的设计大体上可以划分为 3 个部分——系统设计、逻辑设计和电路设计。系统设计涉及将整个系统划分成若干子系统并确定每个子系统的特性。例如,数字计算机的系统设计涉及内存单元、运算单元及输入输出设备的数量及类型的确定,以及这些子系统之间的互连及控制方式的确定。逻辑设计涉及如何将基本的逻辑功能块互连起来以实现特定的功能。例如,确定如何将所需的逻辑门及触发器互连起来实现一个二进制加法器。电路设计涉及如何确定特定部件之间的连接,例如连接寄存器、二极管和晶体管来形成一个门电路、触发器或其他逻辑功能块。当代大多数电路设计采用集成电路的形式,也就是使用适当的计算机辅助工具在一个硅片上设计和互连各个部件。本书主要专注于讲述逻辑设计及理解逻辑设计过程所必须的基础理论。第 18 章和第 20 章涉及系统设计的某些方面,而逻辑门的电路设计将主要在附录 A 中讨论。



图 1-1 开关电路

数字系统中的许多子系统以开关电路(见图 1-1)的形式出现。开关电路具有一个或多个输入端,一个或多个输出端,输入端和输出端都取离散值。在本书中我们将学习两种类型的开关电路——组合电路及时序电路。在组合电路中,输出值仅与当前的输入值有关,而与以前的输入值无

关。而在时序电路中,输出值既与当前的输入值有关,也与以前的输入值有关。换句话说,为了决定一个时序电路的输出,必须给定一系列的输入值。因为时序电路必须记住以前的输入值序列的某些信息,因此有人说它具有记忆功能,但组合电路并没有记忆功能。通常时序电路由组合电路及附加的存储元件组成。组合电路比时序电路简单,我们将首先学习组合电路。

构成组合电路的基本功能块是逻辑门。逻辑设计者必须确定如何将这此逻辑门互连起来以便于将电路的输入信号转变成期望的输出信号。这些输入信号与输出信号之间的关系可以使用布尔代数理论数学化地描述出来。本书的第 2 章和第 3 章介绍了布尔代数中的基本定律与定理,并展示了如何用布尔代数描述逻辑门电路的行为。

从给定的问题描述开始,组合逻辑电路设计的第一步是推导出一个真值表或代数化的逻辑等式,该真值表或等式可以把电路输出描述为电路输入的一个函数(第 4 章)。为了设计一个可以经济地实现此函数的电路,描述该电路输出的逻辑等式必须经过化简。我们将在第 3 章讲述代数化简方法,其他化简方法(卡诺图和奎因-麦克拉斯基方法)我们将在第 5 章和第 6 章介绍。在第 7 章我们讲述如何使用几种门电路实现化简过的逻辑等式,另一种使用可编程逻辑器件的设计过程我们在第 9 章讨论。

时序电路设计中的基本存储元件被称为触发器(第 11 章)。这些触发器与门电路经过互连后可以形成计数器和寄存器(第 12 章)。通常时序电路分析中所用的时序图、状态图及状态表等方法将在第 13 章介绍。设计时序开关电路的第一步是构造用于描述输入输出序列之间关系的状态图或状态表(第 14 章)。第 15 章介绍如何由状态表或状态图得到由逻辑门和触发器组成的电路。用可编程逻辑实现时序电路的方法将在第 16 章讨论。在第 18 章

中,组合电路及时序电路设计技术被用于实现完成二进制加法、乘法及除法功能的系统。本书中设计的时序电路被为同步时序电路,因为这些电路使用一个共同的被称为时钟的时间信号,该信号用来同步存储元件的操作。

第 10、17 和 20 章介绍如何在组合逻辑、时序逻辑和数字系统设计中使用硬件描述语言 VHDL。VHDL 用于对数字硬件进行描述、模拟和合成。在编写好 VHDL 代码之后,设计者可以使用计算机辅助设计软件对这些硬件描述进行编译,从而完成数字逻辑的设计。这使得我们无需根据门电路和触发器手工设计所有电路的细节就可以完成复杂的设计过程。

数字系统中所使用的开关器件通常是具有两种状态的器件,也就是说,可以认为它们的输出值只能取两个不同的离散值。开关器件的例子有继电器、二极管和晶体管。继电器可以认为只有两种状态——关闭或打开,这取决于线圈中是否有电流流过。二极管可以处于导通状态或非导通状态。晶体管可以处于关断状态或饱和状态,相应输出为高电平或低电平。当然晶体管也可以作为线性放大器件使用,相应的输出电压在一个连续的范围内变化。但在数字应用中,若把晶体管用做两状态器件操作的话可以获得更高的可靠性。由于大多数开关器件的输出只能取两个不同的值,因此在数字系统内部使用二进制也就是很自然的事了。基于这个原因,在继续学习开关电路设计之前,我们先讨论二进制数字及数制系统。

1.2 数制系统与转换

当我们写一个十进制数值(以 10 为基)时,我们在采用一种位置相关方法:每个数字要被某个适当的 10 的幂次相乘,这个幂次取决于该数字在该数值中的位置。例如:

$$953.78_{10} = 9 \times 10^2 + 5 \times 10^1 + 3 \times 10^0 + 7 \times 10^{-1} + 8 \times 10^{-2}$$

同理,对于一个二进制数(以 2 为基),每个二进制数字要被某个适当的 2 的幂次相乘:

$$\begin{aligned} 1011.11_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ &= 8 + 0 + 2 + 1 + \frac{1}{2} + \frac{1}{4} = 11 \frac{3}{4} = 11.75_{10} \end{aligned}$$

注意: 二进制小数点将 2 的正数幂次与负数幂次分开,就像十进制小数点将十进制数中的正数幂次与负数幂次分开一样。

任何一个正整数 $R(R > 1)$ 可以被选用作为数制系统的根或基数。若基数为 R ,则共使用 R 个数字 $(0, 1, \dots, R-1)$ 。例如,若 $R=8$,则使用的数字是 0, 1, 2, 3, 4, 5, 6 和 7。一个用位置相关法写出的数值可以用一系列的 R 幂次扩展式表示。例如:

$$\begin{aligned} N &= (a_4 a_3 a_2 a_1 a_0 \cdot a_{-1} a_{-2} a_{-3})_R \\ &= a_4 \times R^4 + a_3 \times R^3 + a_2 \times R^2 + a_1 \times R^1 + a_0 \times R^0 \\ &\quad + a_{-1} \times R^{-1} + a_{-2} \times R^{-2} + a_{-3} \times R^{-3} \end{aligned}$$

这里的 a_i 是 R^i 的系数,并且 $0 \leq a_i \leq R-1$ 。如果幂次扩展式中运算时以 10 为基数,那么所得结果就是 N 的十进制等价式,例如:

$$\begin{aligned} 147.3_8 &= 1 \times 8^2 + 4 \times 8^1 + 7 \times 8^0 + 3 \times 8^{-1} = 64 + 32 + 7 + \frac{3}{8} \\ &= 103.375_{10} \end{aligned}$$

幂次扩展式可以应用于向任何进制的转换。例如,将 147_{10} 转换到三进制可以表示为:

$$147_{10} = 1 \times (101)^2 + (11) \times (101)^1 + (21) \times (101)^0$$

式中等号右侧所有数字都是三进制的数字(提示:在三进制中,10表示为101,7表示为21,以此类推),当进行转换时也应该使用三进制运算。如果用手工完成此计算,的确非常不方便。类似地,将 147_{10} 转换成二进制的计算如下:

$$147_{10} = 1 \times (1010)^2 + (100) \times (1010)^1 + (111) \times (1010)^0$$

当采用手工计算时同样非常不方便,但该计算在计算机内部很容易实现,这是因为计算机内部运算都是采用二进制。对于手工计算,通常用幂次扩展式将其他进制数转换成十进制。

当基数大于10时,为了表示一个数字需要10个以上的符号,在这种情况下一般用字符代表大于9的数字。例如在十六进制(以16为基数)中,A代表 10_{10} ,B代表 11_{10} ,C代表 12_{10} ,D代表 13_{10} ,E代表 14_{10} ,F代表 15_{10} 。因此:

$$A2F_{16} = 10 \times 16^2 + 2 \times 16^1 + 15 \times 16^0 = 2560 + 32 + 15 = 2607_{10}$$

接下来我们讨论用除法将十进制整数转换成以 R 进制数的方法。十进制整数 N 的以 R 为基数的等价式可以表示为:

$$N = (a_n a_{n-1} \cdots a_2 a_1 a_0)_R = a_n R^n + a_{n-1} R^{n-1} + \cdots + a_2 R^2 + a_1 R^1 + a_0$$

如果我们用 N 除以 R ,设余数为 a_0 :

$$\frac{N}{R} = a_n R^{n-1} + a_{n-1} R^{n-2} + \cdots + a_2 R^1 + a_1 = Q_1, \quad \text{余数 } a_0$$

接着我们用商 Q_1 除以 R 得:

$$\frac{Q_1}{R} = a_n R^{n-2} + a_{n-1} R^{n-3} + \cdots + a_3 R^1 + a_2 = Q_2, \quad \text{余数 } a_1$$

接着我们用商 Q_2 除以 R 得:

$$\frac{Q_2}{R} = a_n R^{n-3} + a_{n-1} R^{n-4} + \cdots + a_3 = Q_3, \quad \text{余数 } a_2$$

这个过程继续下去直到我们最后得到 a_n 。注意每个除法步骤中所得余数就是我们想要的某位数字,并且最低位的数字最先得到。

【例】 将 53_{10} 转换成二进制。

$$\begin{array}{rll} 2 \overline{)53} & & \\ 2 \overline{)26} & \text{余数} = 1 = a_0 & \\ 2 \overline{)13} & \text{余数} = 0 = a_1 & \\ 2 \overline{)6} & \text{余数} = 1 = a_2 & \\ 2 \overline{)3} & \text{余数} = 0 = a_3 & \\ 2 \overline{)1} & \text{余数} = 1 = a_4 & \\ 0 & \text{余数} = 1 = a_5 & \end{array} \quad 53_{10} = 110101_2$$

我们可以用连续乘以 R 的方法将一个十进制小数转换成 R 进制的等价值。十进制小数 F 可以表示为:

$$F = (.a_{-1} a_{-2} a_{-3} \cdots a_{-m})_R = a_{-1} R^{-1} + a_{-2} R^{-2} + a_{-3} R^{-3} + \cdots + a_{-m} R^{-m}$$

乘以 R 得到:

$$FR = a_{-1} + a_{-2} R^{-1} + a_{-3} R^{-2} + \cdots + a_{-m} R^{-m+1} = a_{-1} + F_1$$

其中, F_1 代表结果的小数部分, a_{-1} 代表结果的整数部分。 F_1 乘以 R 得到:

$$F_1 R = a_{-2} + a_{-3} R^{-1} + \cdots + a_{-m} R^{-m+2} = a_{-2} + F_2$$

接着将 F_2 乘以 R 得到:

$$F_2 R = a_{-3} + \cdots + a_{-m} R^{-m+3} = a_{-3} + F_3$$

这个过程一直进行下去,直到我们得到足够位数的数字,请注意每一步所得整数部分就是我们想要的某位数字,并且我们最先得到最高位。

【例】 将 0.625_{10} 转换为二进制。

$$\begin{array}{rcl} F = .625 & F_1 = .250 & F_2 = .500 \\ \times 2 & \times 2 & \times 2 \\ \hline 1.250 & 0.500 & 1.000 \\ (a_{-1}=1) & (a_{-2}=0) & (a_{-3}=1) \end{array} \quad .625_{10} = .101_2$$

这个转换过程通常不能终止,当该过程不能终止时,表示转换结果是一个循环小数。

【例】 将 0.7_{10} 转换为二进制。

$$\begin{array}{rcl} .7 & & \\ \times 2 & & \\ \hline (1).4 & & \\ \times 2 & & \\ \hline (0).8 & & \\ \times 2 & & \\ \hline (1).6 & & \\ \times 2 & & \\ \hline (1).2 & & \\ \times 2 & & \\ \hline (0).4 & \leftarrow \text{因为前面得出过 } 0.4, \text{ 所以运算过程从这里开始循环} & \\ \times 2 & & \\ \hline (0).8 & & \end{array} \quad 0.7_{10} = 0.10110\ 0110\ 0110\ \dots_2$$

在两个非十进制的其他进制之间转换也可以直接使用刚才给出的转换过程,但是转换时的算术计算必须采用新的基数而不是使用基数 10。通常比较简单的办法是先将其转换成十进制数,然后再将这个十进制数转换成另外一个基数下的数。

【例】 将 231.3_4 转换为以 7 为基的数。

$$231.3_4 = 2 \times 16 + 3 \times 4 + 1 + \frac{3}{4} = 45.75_{10}$$

$$\begin{array}{rcl} 7 \overline{)45} & & .75 \\ 7 \overline{)6} & \text{余数 } 3 & 7 \\ 0 & \text{余数 } 6 & (5).25 \\ & & 7 \\ & & (1).75 \\ & & 7 \\ & & (5).25 \\ & & 7 \\ & & (1).75 \end{array} \quad 45.75_{10} = 63.5151\ \dots_7$$

将二进制数转换成十六进制数(及相反转换)时,可以通过观察这个数字直接转换,因为每个十六进制数字恰好对应于 4 个二进制数字(位)。从二进制数的小数点开始,每 4 个二进制位划分到一个组里,每个组用一个十六进制数进行替换:

$$1001101.010111_2 = \underbrace{0100}_4 \underbrace{1101}_D . \underbrace{0101}_5 \underbrace{1100}_C = 4D.5C_{16} \quad (1-1)$$

如式(1-1)所示,可以根据需要给每个位串的最后加上多余的 0 以使每个组正好为四位。

1.3 二进制运算

因为完成二进制运算的逻辑电路比十进制运算电路要简单得多,因此,数字系统中的算术运算通常采用二进制。二进制运算的处理方法和十进制基本相同,只是加法表与乘法表简单许多。

二进制数的加法表是:

$$\begin{aligned} 0+0 &= 0 \\ 0+1 &= 1 \\ 1+0 &= 1 \\ 1+1 &= 0 \quad \text{给下一列进位为 1} \end{aligned}$$

给某列进位为 1 相当于给该列加 1。

【例】 用二进制计算 $13_{10} + 11_{10}$ 。

$$\begin{array}{r} 1111 \quad \leftarrow \text{进位} \\ 13_{10} = 1101 \\ 11_{10} = \underline{1011} \\ 11000 = 24_{10} \end{array}$$

二进制数的减法表是:

$$\begin{aligned} 0-0 &= 0 \\ 0-1 &= 1 \quad \text{从下一列借位为 1} \\ 1-0 &= 1 \\ 1-1 &= 0 \end{aligned}$$

从某列借位为 1 相当于从该列减 1。

【例】 二进制减法

(a) $1 \leftarrow$ (表示从第三列借位) $\begin{array}{r} 11101 \\ -10011 \\ \hline 1010 \end{array}$	(b) $1111 \leftarrow$ 借位 $\begin{array}{r} 10000 \\ - \quad 11 \\ \hline 1101 \end{array}$	(c) $111 \leftarrow$ 借位 $\begin{array}{r} 111001 \\ - \quad 1011 \\ \hline 101110 \end{array}$
---	--	--

注意: 在第 2 个例子借位是怎样从一列传递到下一列的。为了从第 2 列借 1,我们必须依次从第 3 列借 1,等等。二进制减法运算的另一种方法是采用补码运算,我们将在 1.4 节讨论。

在二进制减法运算中我们有时会搞混淆,可能是因为我们对于十进制减法运算太熟悉,以至于常常忘记两者借位运算的重要不同之处。在进行详细的二进制减法运算分析之前,我们先回顾一下十进制减法运算的借位处理过程。

假设我们从右向左给十进制数字的每列(数字)进行编号(由 0 开始),那么当我们从第 n 列借 1,就意味着从第 n 列减 1 并且给第 $n-1$ 列加 10。因为 $1 \times 10^n = 10 \times 10^{n-1}$,十进制

第2列 第1列

$$\begin{array}{r} 205 \\ - 18 \\ \hline 187 \end{array}$$
$$\begin{aligned}
 205 - 18 &= [2 \times 10^2 + 0 \times 10^1 + 5 \times 10^0] \\
 &\quad - [1 \times 10^1 + 8 \times 10^0] \\
 &\quad \quad \quad \downarrow \quad \quad \quad \downarrow \quad \text{注意从第1列借位} \\
 &= [2 \times 10^2 + (0 - 1) \times 10^1 + (10 + 5) \times 10^0] \\
 &\quad - [1 \times 10^1 + 8 \times 10^0] \\
 &\quad \quad \quad \downarrow \quad \quad \quad \downarrow \quad \text{注意从第2列借位} \\
 &= [(2 - 1) \times 10^2 + (10 + 0 - 1) \times 10^1 + 15 \times 10^0] \\
 &\quad - [1 \times 10^1 + 8 \times 10^0] \\
 &= [1 \times 10^2 + 8 \times 10^1 + 7 \times 10^0] = 187
 \end{aligned}$$
$$\begin{aligned} 0 \times 0 &= 0 \\ 0 \times 1 &= 0 \\ 1 \times 0 &= 0 \\ 1 \times 1 &= 1 \end{aligned}$$
$$\begin{array}{r} 1101 \\ 1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 10001111 = 143_{10} \end{array}$$

请注意：每个部分积是被乘数适当移位后所得，或者为零。

当把某一列的多个二进制数位一起加起来时,该列的和可能会超过 11_2 ,因此向下一列的进位可能会超过 1。例如若某列的二进制数位中包含 5 个 1,那么加起来的的结果是 101_2 ,这意味着该列的加法和为 1,而向下一列的进位为 10_2 。在进行二进制乘法时,避免进位大于 1 的常用方法是一次只加一个部分积,如下例所示:

1111	被乘数
<u>1101</u>	乘数
1111	第1个部分积
0000	第2个部分积
(01111)	第1、2个部分积之和
<u>1111</u>	第3个部分积
(1001011)	加上第3个部分积之后的和
<u>1111</u>	第4个部分积
1100011	最后乘积结果(加上第4个部分积之后的和)

下面的例子阐明了用二进制实现 145_{10} 除以 11_{10} 的运算:

$$\begin{array}{r}
 1101 \\
 1011 \overline{) 1001001} \\
 \underline{1011} \\
 1110 \\
 \underline{1011} \\
 1101 \text{ 商为1101并且余数为10} \\
 \underline{1011} \\
 10
 \end{array}$$

二进制除法与十进制除法相类似,只是更简单一些,因为只有两种可能的商值是 0 和 1。在上面的例子中,如果我们开始将除数(1011)与被除数的最高 4 位(1001)相比较,会发现在不允许出现负数结果的情况下没有办法进行减法,因此,我们将除数右移一位后重新尝试。这一次我们可以从 10010 中减去 1011 并且结果为 111,因此我们在 10010 上面放置第 1 位商值 1。接着将被除数的下一位(0)移下来得到 1110 并且将除数右移一位,再从 1110 中减去 1011 后得到 11,所以第 2 个位商值为 1。当我们移下后面的被除数位后结果是 110,它不能减去移位后的除数,因此第 3 个商位为 0。我们移下最后的被除数位并且从 1101 中减去 1011 后,得到最后的余数 10 并且最后一位的商值为 1。

1.4 负数的表示

至今为止我们使用的是非负正数,既能表示正数,又能表示负数的最常用方法有原码(sign and magnitude)、补码(2's complement)和反码(1's complement)。在这 3 种方法中,最左边的一位若为 0 表示正数,若为 1 表示负数。如下面所讨论,当用 n 位表示一个数时,原码和反码能表示的数据范围为 $-(2^{(n-1)}-1) \sim +(2^{(n-1)}-1)$,并且 0 有两种表示格式:正 0 和负 0。而补码能表示的数据范围为 $-2^{(n-1)} \sim +(2^{(n-1)}-1)$,并且只有正 0。对于某种运算,例如加法或者减法,若两个数的运算结果超出了所能表示的数据范围,我们称为溢出(overflow)。

1.4.1 原码

在 n 位原码系统中,一个数字表示为 1 个符号位后面跟 $n-1$ 个数值位的形式。符号位为 0 表示正数,为 1 表示负数。数值位代表该数的绝对值。 $n-1$ 位绝对值可表示 $0 \sim (2^{(n-1)}-1)$ 之间的数,再加上符号位,因此 n 位原码能表示的数据范围为 $(-(2^{(n-1)}-1) \sim +(2^{(n-1)}-1))$,包括正 0 和负 0。 $n=4$ 的原码例子如表 1-1 所示。例如,0011 表示 +3,1011 表示 -3。注意 1000 表示负 0。

表 1-1 带符号的二进制整数(字长: $n=4$)

+N	正整数(所有系统)	-N	负整数		
			原码	补码	反码
+0	0000	-0	1000	——	1111
+1	0001	-1	1001	1111	1110
+2	0010	-2	1010	1110	1101
+3	0011	-3	1011	1101	1100
+4	0100	-4	1100	1100	1011
+5	0101	-5	1101	1011	1010
+6	0110	-6	1110	1010	1001
+7	0111	-7	1111	1001	1000
		-8	——	1000	——

设计直接用原码进行算术运算的逻辑电路很麻烦。一种方法是先将原码转换为补码(或者反码)进行运算,运算完成后再将结果转换回原码。

1.4.2 补码

在补码系统中,一个正数 N 表示为 0 后面跟上该数的绝对值,和该数的原码表示一样。但一个负数 $-N$ 表示为 N 的补数 N^* 。当字长为 n 时,正整数 N 的补码定义如下:

$$N^* = 2^n - N \quad (1-2)$$

(注意:上式中所有数字 N 、 2^n 和 n^* 都当做无符号正数;当采用二进制时, 2^n 需要用 $n+1$ 位表示。)表 1-1 给出了 $n=4$ 时的补码表示。在表 1-1 中, $-1 \sim -7$ 的补码可以通过计算 $1 \sim 7$ 的补数求取(即从 16 减去该数)。例如,5 的补数为 $16-5=11$,或者用二进制表示如下: $(10000)-(0101)=(1011)$ 。通过减法运算,4 个二进制的组合可以表示数字 $-7, \dots, -1, 0, 1, \dots, 7$ 。唯一未被使用的组合是 1000。由于 1000 最左一位为 1,因此应该是个负数。注意到一个负数的绝对值等于该数的补数,为了求该数的绝对值,我们可以从式(1-2)得:

$$N = 2^n - N^* \quad (1-3)$$

将式(1-3)应用到 1000 得: $(10000)-(1000)=(1000)$,或者用十进制表示为 $16-8=8$ 。因此,1000 表示 -8 的补码。通常在 n 位补码表示中,1 后面所有位为 0 表示 $-2^{(n-1)}$ 。

对二进制数直接应用式(1-2)需要做 $n+1$ 位减法。将式(1-2)改写为下式就可以避免这一点:

$$N^* = (2^n - 1 - N) + 1$$

在二进制中, $2^n - 1$ 代表 n 位全为 1。用一个全 1 的数减去另一个数不会产生借位, 通过将 0 变为 1 以及将 1 变为 0 即可完成该减法(即简单地对 N 一位一位取反)。例如, 如果 $n=7$ 并且 $N=0101100$, 则:

$$\begin{array}{r} 2^n - 1 = 1111111 \\ - 0101100 \\ \hline 1010011 \\ + 0000001 \\ \hline N^* = 1010100 \end{array}$$

N^* 可以通过将 N 每位取反再加上 1 得到。另一种求取 N 的补码的方法是: 从 N 的右边开始, 使所有的 0 和最右边的第一个 1 保持不变, 将这个 1 左边的每个位都取反得到。在前面的例子中, N 右边的 100 保持不变, 而将左边的 0101 每位取反即可。

1.4.3 补码的加法

n 位带符号的二进制数的加法运算在补码系统中可以直接计算, 加法中的运算过程就像所有数是正数一样, 并且符号位产生的进位都需要忽略。除了发生溢出的情况外, 这样计算总能得到正确结果。当字长为 n 位时, 正确的运算和(含有符号位)若需要多于 n 位才能表示, 我们称为溢出。以下 6 点解释了当 $n=4$ 时不同情况下出现的结果。

(1) 两个正数相加, 和小于 2^{n-1} 。

$$\begin{array}{r} +3 \quad 0011 \\ +4 \quad 0100 \\ \hline +7 \quad 0111 \end{array} \quad (\text{结果正确})$$

(2) 两个正数相加, 和大于等于 2^{n-1} 。

$$\begin{array}{r} +5 \quad 0101 \\ +6 \quad 0110 \\ \hline 1011 \end{array} \quad \leftarrow \text{由于发生溢出, 结果错误 (+11 带符号位共需要 5 位才能表示)}$$

(3) 正数与负数相加(负数的绝对值较大)。

$$\begin{array}{r} -5 \quad 0101 \\ -6 \quad 1010 \\ \hline -1 \quad 1111 \end{array} \quad (\text{结果正确})$$

(4) 与第(3)种情况相同, 只是正数的绝对值较大。

$$\begin{array}{r} +5 \quad 1011 \\ +6 \quad 0110 \\ \hline +1 \end{array} \quad (1)0001 \leftarrow \text{符号位产生的进位忽略, 结果正确 (没有溢出)}$$

(5) 两个负数的加法, 和的绝对值小于等于 2^{n-1} 。

$$\begin{array}{r} -3 \quad 1101 \\ -4 \quad 1100 \\ \hline -7 \end{array} \quad (1)1001 \leftarrow \text{最后的进位忽略, 结果正确 (没有溢出)}$$

(6) 两个负数的相加, 和的绝对值大于 2^{n-1} 。

$$\begin{array}{r} -5 \quad 1011 \\ -6 \quad 1010 \\ \hline \end{array}$$

(1)0101 ← 发生溢出, 结果错误 (—11 带符号位共需要 5 位才能表示)

注意: 溢出条件((2)和(6))很容易被检测出来。因为, 在(2)中两个正数相加的结果为负数, 而在(6)中两个负数相加的结果为正数(对于 4 位字长来讲)。

(4)及(5)中丢弃掉符号位的进位可以产生正确的结果, 其证明如下:

在(4)中: $-A+B$ (当 $B>A$ 时)

$$A^* + B = (2^n - A) + B = 2^n + (B - A) > 2^n$$

丢弃最后的进位等价于减去 2^n , 因此结果为 $(B-A)$ 是正确的。

在(5)中: $-A-B$ (当 $A+B \leq 2^{n-1}$)

$$A^* + B^* = (2^n - A) + (2^n - B) = 2^n + 2^n - (A + B)$$

丢弃最后的进位得到 $2^n - (A+B) = (A+B)^*$, 这是一 $(A+B)$ 的正确表示形式。

1.4.4 反码

在反码系统中, 一个负数 $-N$ 表示为 N 的反数 $\bar{N}$ 。 $\bar{N}$ 定义如下:

$$\bar{N} = (2^n - 1) - N \quad (1-4)$$

如前所述, $(2^n - 1)$ 由 n 个 1 组成, 用 1 减去一个位等于该位的反。因此, N 的反数可以将 N 逐位取反得到。表 1-1 给出了 $n=4$ 时的反码表示。注意: 0000 的反数为 1111, 这表示负 0 的反码。反码中 0 有两种表示格式, 就像原码一样。

1.4.5 反码的加法

反码的加法与补码加法相类似, 只是最后产生的进位不是丢弃掉而是再次加到 n 位和的最右位上, 这被称为循环进位(end-around carry)。正数反码加法和前面补码加法示例中(见 1.4.3 节)前 2 点所示方法相同。下面给出了其余的反码加法情况($n=4$)。

(3) 正数与负数相加(负数绝对值较大)。

$$\begin{array}{r} +5 \quad 0101 \\ -6 \quad 1001 \\ \hline -1 \quad 1110 \quad (\text{结果正确}) \end{array}$$

(4) 与(3)相同, 只是正数绝对值较大。

$$\begin{array}{r} -5 \quad 1010 \\ +6 \quad 0110 \\ \hline (1) \quad 0000 \\ \quad \quad \quad \downarrow \quad 1 \quad (\text{循环进位}) \\ \quad \quad \quad 0001 \quad (\text{结果正确, 没有溢出}) \end{array}$$

(5) 两个负数相加, 结果的绝对值小于 2^{n-1} 。

$$\begin{array}{r}
 -3 \quad 1100 \\
 -4 \quad 1011 \\
 \hline
 (1) \quad 0111 \\
 \quad \quad \quad \rightarrow 1 \quad (\text{循环进位}) \\
 \hline
 \quad \quad 1000 \quad (\text{结果正确, 没有溢出})
 \end{array}$$

(6) 两个负数相加, 结果的绝对值大于等于 2^{n-1} 。

$$\begin{array}{r}
 -5 \quad 1010 \\
 -6 \quad 1001 \\
 \hline
 (1) \quad 0111 \\
 \quad \quad \quad \rightarrow 1 \quad (\text{循环进位}) \\
 \hline
 \quad \quad 0100 \quad (\text{因为溢出, 结果错误})
 \end{array}$$

再次注意: (6) 中的溢出情况很容易检测, 因为两个负数相加的结果为正数。

(4)、(5) 中的循环进位方法可以产生正确结果, 其证明如下:

在(4)中: $-A+B$ (此处 $B>A$)

$$\bar{A} + B = (2^n - 1 - A) + B = 2^n + (B - A) - 1$$

循环进位等价于减去 2^n 并加 1, 因此结果为 $(B-A)$, 其结果是正确的。

在(5)中: $-A-B$ ($A+B < 2^{n-1}$)

$$\bar{A} + \bar{B} = (2^n - 1 - A) + (2^n - 1 - B) = 2^n + [2^n - 1 - (A + B)] - 1$$

经过循环进位后结果为 $2^n - 1 - (A + B) = \overline{(A + B)}$, 这正好是 $-(A+B)$ 的正确反码表示形式。

下面的例子阐述了当字长 $n=8$ 时的反码与补码加法运算:

1. 用反码求 -11 与 -20 的和

$$+11 = 00001011, \quad +20 = 00010100。$$

逐位取反得:

$$-11 \text{ 表示为 } 11110100, \quad -20 \text{ 表示为 } 11101011。$$

$$\begin{array}{r}
 11110100 \quad (-11) \\
 11101011 \quad +(-20) \\
 \hline
 (1) \quad 11011111 \\
 \quad \quad \quad \rightarrow 1 \quad (\text{循环进位}) \\
 \hline
 \quad \quad 11100000 = -31
 \end{array}$$

2. 用补码求 -8 与 $+19$ 的和

$$+8 = 00001000$$

将第一个为 1 的位的左边所有位取反, 因此 -8 的补码表示为 11111000 。

$$\begin{array}{r}
 11111000 \quad (-8) \\
 00010011 \quad +19 \\
 \hline
 (1) 00001011 = +11 \\
 \quad \quad \quad \uparrow \quad (\text{丢弃最后的进位})
 \end{array}$$

注意: 在这两个例子中, 加法运算在最左边的位上都产生了一个进位, 但并没有产生溢出, 这是因为加法结果可以用 8 位正确表示 (包括符号位)。在对两个 n 位的带符号二进制

数(反码或补码)相加产生 n 位和的运算中,判断溢出的一个通用规则是:

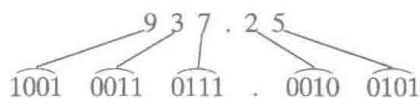
若两正数相加产生负数结果,或两负数相加产生正数结果,则结果溢出。

补码加法运算判断溢出的另一方法如下:

有溢出产生当且仅当运算时符号位的进位输出不等于符号位的进位输入^①。

1.5 二进制编码

虽然大多数计算机内部使用二进制数,但输入输出设备通常使用十进制数。由于大多数逻辑电路只接受二值信号,因此一个十进制数必须表示为二进制信号形式。在最简单的二进制编码中,每个十进制数字用它所对应的二进制代码替换,例如 937.25 表示为:



这种表示形式称为二进制编码的十进制(Binary Coded Decimal, BCD),或直接称为 8-4-2-1BCD 编码。注意:编码结果与把这个数整体转换为二进制数所得形式完全不同。由于只有 10 个十进制数字,因此在 BCD 编码中,1011~1111 是无效的 BCD 编码。

表 1-2 表示了 10 个十进制数字的几种可能的二进制编码表。目前还存在许多其他的编码方法,因为对于一种合法编码方法的唯一要求是每个十进制数字的二进制表示有唯一二进制组合。在将一个十进制数转换成编码形式时,只需将每个十进制数字替换成对应的编码即可。因此 937 在余 3 码中表示为 1100 0110 1010。8-4-2-1(BCD)码和 6-3-1-1 码是有权码的例子。4 位的有权码具有这样的特性:若权重分别是 w_3 、 w_2 、 w_1 和 w_0 ,则编码 $a_3 a_2 a_1 a_0$ 表示的十进制数字 N 是:

$$N = w_3 a_3 + w_2 a_2 + w_1 a_1 + w_0 a_0$$

表 1-2 十进制数字的二进制编码

十进制数	8-4-2-1 编码(BCD)	6-3-1-1 编码	余 3 码	5 中 2 位 为 1 码	格雷 (Gray)码
0	0000	0000	0011	00011	0000
1	0001	0001	0100	00101	0001
2	0010	0011	0101	00110	0011
3	0011	0100	0110	01001	0010
4	0100	0101	0111	01010	0110
5	0101	0111	1000	01100	1110
6	0110	1000	1001	10001	1010
7	0111	1001	1010	10010	1011
8	1000	1011	1011	10100	1001
9	1001	1100	1100	11000	1000

① 即最高数值位的进位输出。(译者注)

例如,6-3-1-1 编码的权重分别是 $w_3=6$ 、 $w_2=3$ 、 $w_1=1$ 和 $w_0=1$,因此二进制代码 1011 表示的十进制数字是:

$$N=6 \cdot 1+3 \cdot 0+1 \cdot 1+1 \cdot 1=8$$

余 3 码可以通过给 8-4-2-1 码中每个编码加 3(0011)而求得。5 中 2 位为 1 码具有这样的特性:每个合法的编码组合的 5 位中正好有 2 位为 1。该编码具备很有用的错误检测功能,若电路的误操作使得编码中的任何一位发生改变,则编码中 1 的个数不再是 2 个。表 1-2 中给出了格雷码的一种例子。格雷码具有这样的特性:连续的两个十进制数字的格雷码中只有 1 个位不同。例如 6 和 7 的编码中只有第 4 位不同,9 和 0 的编码只有第 1 位不同。格雷码通常用于模拟量的转换中,例如将转轴的位置量转换成数字形式。在这种情况下,模拟量的微小变化只改变编码中的一位,这比同时改变两位或多位具有更高的操作可靠性。格雷码和 5 中 2 位为 1 码都不是权重码。通常情况下,当使用非权重码时,我们不能通过简单公式计算来求出权重码对应的十进制数字。

许多计算机应用需要处理的数据包含有数字、字母以及其他符号例如标点符号等。为了将这样的字符数据与计算机内部格式之间相互转换或将其在计算机内部存储,每一种符号必须表示成某种二进制编码。一种通用的字符编码是 ASCII 编码(即美国标准信息交换编码),这是一种 7 位编码,因此共有 2^7 (128)种不同的二进制组合可用来表示字母、数字以及其他符号。表 1-3 显示了 ASCII 编码的一部分,表中未列出的编码组合用于特殊的控制功能,例如“换页”(form feed)或“传送完成”(end of transmission)。单词“Start”的 ASCII 表示形式为:

1010011 1110100 1100001 1110010 1110100
S t a r t

表 1-3 ASCII 编码

ASCII 编码								ASCII 编码								ASCII 编码							
字符	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	字符	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	字符	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
空格符	0	1	0	0	0	0	0	@	1	0	0	0	0	0	0	'	1	1	0	0	0	0	0
!	0	1	0	0	0	0	1	A	1	0	0	0	0	0	1	a	1	1	0	0	0	0	1
"	0	1	0	0	0	1	0	B	1	0	0	0	0	1	0	b	1	1	0	0	0	1	0
#	0	1	0	0	0	1	1	C	1	0	0	0	0	1	1	c	1	1	0	0	0	1	1
\$	0	1	0	0	1	0	0	D	1	0	0	0	1	0	0	d	1	1	0	0	1	0	0
%	0	1	0	0	1	0	1	E	1	0	0	0	1	0	1	e	1	1	0	0	1	0	1
&	0	1	0	0	1	1	0	F	1	0	0	0	1	1	0	f	1	1	0	0	1	1	0
'	0	1	0	0	1	1	1	G	1	0	0	0	1	1	1	g	1	1	0	0	1	1	1
(	0	1	0	1	0	0	0	H	1	0	0	1	0	0	0	h	1	1	0	1	0	0	0
)	0	1	0	1	0	0	1	I	1	0	0	1	0	0	1	i	1	1	0	1	0	0	1
*	0	1	0	1	0	1	0	J	1	0	0	1	0	1	0	j	1	1	0	1	0	1	0
+	0	1	0	1	0	1	1	K	1	0	0	1	0	1	1	k	1	1	0	1	0	1	1
,	0	1	0	1	1	0	0	L	1	0	0	1	1	0	0	l	1	1	0	1	1	0	0
-	0	1	0	1	1	0	1	M	1	0	0	1	1	0	1	m	1	1	0	1	1	0	1
.	0	1	0	1	1	1	0	N	1	0	0	1	1	1	0	n	1	1	0	1	1	1	0
/	0	1	0	1	1	1	1	O	1	0	0	1	1	1	1	o	1	1	0	1	1	1	1

续表

ASCII 编码								ASCII 编码								ASCII 编码							
字符	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	字符	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	字符	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
0	0	1	1	0	0	0	0	P	1	0	1	0	0	0	0	p	1	1	1	0	0	0	0
1	0	1	1	0	0	0	1	Q	1	0	1	0	0	0	1	q	1	1	1	0	0	0	1
2	0	1	1	0	0	1	0	R	1	0	1	0	0	1	0	r	1	1	1	0	0	1	0
3	0	1	1	0	0	1	1	S	1	0	1	0	0	1	1	s	1	1	1	0	0	1	1
4	0	1	1	0	1	0	0	T	1	0	1	0	1	0	0	t	1	1	1	0	1	0	0
5	0	1	1	0	1	0	1	U	1	0	1	0	1	0	1	u	1	1	1	0	1	0	1
6	0	1	1	0	1	1	0	V	1	0	1	0	1	1	0	v	1	1	1	0	1	1	0
7	0	1	1	0	1	1	1	W	1	0	1	0	1	1	1	w	1	1	1	0	1	1	1
8	0	1	1	1	0	0	0	X	1	0	1	1	0	0	0	x	1	1	1	1	0	0	0
9	0	1	1	1	0	0	1	Y	1	0	1	1	0	0	1	y	1	1	1	1	0	0	1
:	0	1	1	1	0	1	0	Z	1	0	1	1	0	1	0	z	1	1	1	1	0	1	0
;	0	1	1	1	0	1	1	[	1	0	1	1	0	1	1	{	1	1	1	1	0	1	1
<	0	1	1	1	1	0	0	\	1	0	1	1	1	0	0		1	1	1	1	1	0	0
=	0	1	1	1	1	0	1	]	1	0	1	1	1	0	1	}	1	1	1	1	1	0	1
>	0	1	1	1	1	1	0	^	1	0	1	1	1	1	0	~	1	1	1	1	1	1	0
?	0	1	1	1	1	1	1	—	1	0	1	1	1	1	1	删除符	1	1	1	1	1	1	1

习题

1.1 将下列数字转换成十六进制数,再转换成二进制数。

(a) 757.25_{10}

(b) 123.17_{10}

(c) 356.89_{10}

(d) 1063.5_{10}

1.2 将下列数字转换成八进制,再转换成十六进制。最后将两者转换成十进制,并验证它们是相同的。

(a) 111010110001.011_2

(b) 10110011101.11_2

1.3 将 $3BA.25_{16}$ 转换成六进制(所有运算用十进制完成)。

1.4 (a) 将 1457.11_{10} 转换成十六进制,十六进制小数点后舍入到 2 位。

(b) 将得到的结果转换成二进制,再转换成八进制。

(c) 设计一种方案可以直接将十六进制转换成四进制,然后将你的结果转换成四进制。

(d) 将 $DEC. A_{16}$ 转换成十进制。

1.5 用二进制完成下列数的加法、减法及乘法运算。

(a) 1111 和 1010

(b) 110110 和 11101

(c) 100100 和 10110

1.6 用二进制完成减法运算,在需要被借位的列上放置一个 1。

(a) $11110100 - 1000111$

(b) $1110110 - 111101$

(c) $10110010 - 111101$

1.7 设字长为 6 位(包含符号位),负数用补码表示完成下列数的二进制加法运算,并指出是否发生了溢出。

(a) $21 + 11$

(b) $(-14) + (-32)$

(c) $(-25) + 18$

(d) $(-12)+13$ (e) $(-11)+(-21)$

负数用反码表示,重新完成上述(a)、(c)、(d)和(e)的计算。

1.8 某计算机的字长为8位(包含符号位),若负数用补码表示,该计算机可以存储的整数范围是多少?若负数用反码表示,该计算机可以存储的整数范围又是多少?(结果用十进制表示。)

1.9 构造权重为7-3-2-1的编码并使用这种编码表示3659。

1.10 将下列数转换成十六进制,再转换成二进制。

(a) 1305.375_{10} (b) 111.33_{10} (c) 301.12_{10} (d) 1644.875_{10}

1.11 将下列数转换成八进制,再转换成十六进制,最后将八进制和十六进制结果转换成十进制,并检验它们是否相同。

(a) 101111010100.101_2 (b) 100001101111.01_2

1.12 (a) 将 375.54_8 转换成三进制数(所有运算采用十进制)。

(b) 将 384.74_{10} 转换成四进制数。

(c) 将 $A52.A4_{11}$ 转换成九进制数(所有运算采用十进制)。

1.13 将 544.1_9 转换成十六进制数,再转换成二进制数。

1.14 将十进制数 97.7_{10} 转换成结果完全相同的下列进制数表示。转换结果的小数部分需要用无限循环表示。请给出推导步骤。

(a) 二进制 (b) 八进制 (c) 十六进制

(d) 三进制 (e) 五进制

1.15 设计一种方案用于直接将三进制数转换成九进制数。使用你设计的方法将下面的数字转换成九进制数: 1110212.20211_3 。

1.16 将下列十进制转换成八进制,然后转换成二进制。

(a) $2983\frac{63}{64}$ (b) 93.70 (c) $2983\frac{31}{32}$ (d) 109.30

1.17 用二进制完成下列数的加法、减法和乘法运算。

(a) 1111 和 1001 (b) 1101001 和 110110 (c) 110010 和 11101

1.18 用二进制完成减法运算,在需要被借位的列上放置一个1。

(a) $10100100-01110011$ (b) $10010011-01011001$

(c) $11110011-10011110$

1.19 用二进制完成下列数的除法运算。

(a) $11101001\div 101$ (b) $110000001\div 1110$ (c) $1110010\div 1001$

用二进制将商和除数相乘再加上余数来检查结果是否正确。

1.20 用二进制完成下列数的除法运算。

(a) $10001101\div 110$ (b) $110000011\div 1011$ (c) $1110100\div 1010$

1.21 假设用3位数字表示一个正整数并且下列运算正确。请确定所采用的基数。是否有运算发生溢出?

(a) $654+013=000$ (b) $024+043+013+033=233$

(c) $024+043+013+033=201$

1.22 二进制需要最少多少位才能大致与十进制数 0.6117_{10} 相等,并且二进制要具有

相同或者更好的精度?

1.23 将 $0.363636\dots_{10}$ 准确地转换为八进制数。

1.24 (a) 证明用下述方法可以将一个 b 进制数转换成 b^3 进制数: 将该 b 进制数以小数点为界同时向左右两边每 3 个相邻位构成一组, 再将每个组转换成 b^3 进制数中的数字位。(提示: 将 b 进制数表示成幂次展开式。)

(b) 证明用下述方法可以将一个 b^3 进制数转换成 b 进制数: 将该 b^3 进制数以小数点为界每个数字转换成 3 位的 b 进制数字, 再将转换结果从左向右相串接。

1.25 (a) 指出数字 $(5-1)$ 、 (5^2-1) 和 (5^3-1) 在五进制中如何表示。

(b) 将(a)的答案归纳并指出如何将 (b^n-1) 表示为 b 进制格式。此处的 b 可以是任何大于 1 的整数, n 可以是任意大于 0 的整数。给出数学推导过程。

1.26 (a) 设基数 $b>2$, 证明数字 121_b 是一个完美平方数(也就是该数是某个数的平方)。

(b) 对于数字 12321_b 重新完成(a)中的证明, 此处 $b>3$ 。

(c) 对于数字 14641_b 重新完成(a)中的证明, 此处 $b>6$ 。

(d) 对于数字 1234321_b 重新完成(a)中的证明, 此处 $b>4$ 。

1.27 (a) 将 $(0.12)_3$ 转换为六进制小数。

(b) 将 $(0.375)_{10}$ 转换为八进制小数。

(c) 令 $N=(0.a_{-1}a_{-2}\dots a_{-m})_R$ 是一个 R 进制小数并且最多有 m 个非零小数位, 确定将 N 可以转换为 S 进制小数 $N=(0.b_{-1}b_{-2}\dots b_{-n})_S$, 且转换结果包含有限个非零小数位的充分必要条件。(提示: (a) 给出了一个例子, 并注意 $(a_{-1}R^{-1}+a_{-2}R^{-2}+\dots+a_{-m}R^{-m})S^n$ 必须是一个整数。)

(d) 总结(a), 若 N 是某个特定的而不是所有的基数 R 下的小数 $N=(0.b_{-1}b_{-2}\dots b_{-m})_R$, 确定将 N 可以转换为 S 进制小数且转换结果包含有限个非零小数位的充分必要条件。

1.28 构造权重为 4-3-2-1 的编码表, 并用这种编码表示数字 9154。

1.29 是否能构造出权重为 5-3-1-1 的编码表? 是否能构造出权重为 6-4-1-1 的编码表? 证明你的答案。

1.30 是否能构造出权重为 5-4-1-1 的编码表? 是否能构造出权重为 6-3-2-1 的编码表? 证明你的答案。

1.31 为十进制数字构造权重为 6-2-2-1 的编码表。采用这种编码时, 1100 0011 表示什么数字?

1.32 为十进制数字构造权重为 5-2-2-1 的编码表。采用这种编码时, 1110 0110 表示什么数字?

1.33 为十二进制数构造权重为 7-3-2-1 的编码表, 用此编码写出 B4A9 的表示形式。

1.34 (a) 是否可以构造十进制数构造权重为负数的带权编码表? 例如权重为 8, 4, -2, -1。请构造该编码。

(b) 已知十进制数字 d 在上述编码表中的编码, 如何求取 $9-d$ 的编码?

1.35 将下列数转换成十六进制数, 然后给出这个十六进制数(包含十六进制小数点)的 ASCII 编码。

(a) 222.22_{10}

(b) 183.81_{10}

1.36 用下列数重新完成 1.7 题的要求。

(a) $(-10) + (-11)$

(b) $(-10) + (-6)$

(c) $(-8) + (-11)$

(d) $11 + 9$

(e) $(-11) + (-4)$

1.37 由于 $A - B = A + (-B)$, 因此减去一个带符号的数可以通过加上这个数的补来实现。假定负数用反码表示, 通过将减数的补加到被减数完成下列每对 5 位二进制数的减法运算, 并指出是否发生溢出。将负数用补码表示, 再次完成每对数的减法运算。

(a)
$$\begin{array}{r} 01001 \\ -11010 \\ \hline \end{array}$$

(b)
$$\begin{array}{r} 11010 \\ -11001 \\ \hline \end{array}$$

(c)
$$\begin{array}{r} 10110 \\ -01101 \\ \hline \end{array}$$

(d)
$$\begin{array}{r} 11011 \\ -00111 \\ \hline \end{array}$$

(e)
$$\begin{array}{r} 11100 \\ -10101 \\ \hline \end{array}$$

1.38 针对下列每对数字重新完成 1.37 题的要求。

(a)
$$\begin{array}{r} 11010 \\ -10100 \\ \hline \end{array}$$

(b)
$$\begin{array}{r} 01011 \\ -11000 \\ \hline \end{array}$$

(c)
$$\begin{array}{r} 10001 \\ -01010 \\ \hline \end{array}$$

(d)
$$\begin{array}{r} 10101 \\ -11010 \\ \hline \end{array}$$

1.39 (a) 设两个反码数 $A = 101010, B = 011101$, 执行下列运算并判断是否发生溢出。

(i) $A + B$

(ii) $A - B$

(b) 设 A 和 B 都是补码数, 重新完成(a)中的运算。

1.40 (a) 假设下列数据都是反码数, 求每个数的反数并给出每个数与其反数的真值。

(i) 0000000

(ii) 1111111

(iii) 00110011

(iv) 1000000

(b) 假设(a)中的数都是补码数, 求每个数的补数并给出每个数与其补数的真值。

1.41 将一个 20 进制数 $d_{n-1}d_{n-2}\cdots d_1d_0$ 转换为十进制的另一种方法是: 给每个 d_i 乘以 2^i 再在右边补 i 个 0, 最后将所有结果相加。

(a) 使用该算法将 $GA7_{20}$ 转换为十进制数(G_{20} 即就是 16_{10})。

(b) 证明该算法的正确性。

(c) 思考将二十进制小数 $0.d_{-1}d_{-2}\cdots d_{-n+1}d_{-n}$ 转换为十进制小数, 按照上述思路给出一种可实现该转换的算法。

(d) 用(c)中给出的算法转换 $0.FA7_{20}$ 。

1.42 设 A 和 B 都是正整数, 考虑采用 n 位补码实现 A 和 B 相加。

(a) 证明当 $(A+B)$ 的绝对值 $< 2^{n-1} - 1$ 时, 将 A 和 B 相加会得到正确的补码表示运算结果。但当 $(A+B)$ 的绝对值 $> 2^{n-1} - 1$ 时, 运算结果是绝对值为 $2^n - (A+B)$ 的负数的补码表示。

(b) 对于 $A \geq B$ 和 $A < B$ 两种情况, 证明 A 和 $(-B)$ 相加总能得到正确结果的补码表示。

(c) 证明当 $A+B$ 的绝对值 $\leq 2^{n-1}$ 并且忽略符号位产生的进位时, 将 $(2^n - A)$ 和 $(2^n - B)$ 相加会得到 $-(A+B)$ 正确的补码表示结果。再证明当 $A+B$ 的绝对值 $> 2^{n-1}$ 时, 运算结果不正确, 结果为 $2^n - (A+B)$ 。

1.43 设 A, B 都是整数并且采用 n 位反码表示格式, 证明在不发生溢出的情况下, 将 A 和 B 相加再进行循环进位相加总能得到正确的反码结果。证明时考虑下面 4 种情况: A 和 B 都是正数; A 是正数 B 是负数且 A 绝对值大于 B 绝对值; A 是正数 B 是负数且 A 绝

对值小于或者等于 B 绝对值； A 和 B 都是负数。

1.44 证明当采用补码进行加法运算时,加法溢出的条件是当且仅当符号位的进位输出情况与符号位的进位输入情况不同。证明时考虑下面 3 种情况:两个正数相加;两个负数相加;两个不同符号数相加。

1.45 重新描述 1.44 题中检测溢出的方法并使该方法可以用于反码相加。

1.46 令 $B=b_{n-1}b_{n-2}\cdots b_1b_0$ 是一个 n 位补码数,证明 B 的十进制真值是 $-b_{n-1}2^{n-1}+b_{n-2}2^{n-2}+b_{n-3}2^{n-3}+\cdots+b_12+b_0$ 。(提示:分别考虑正数($b_{n-1}=0$)和负数($b_{n-1}=1$),并注意到将一个负数的每个位用 1 相减(即每个位取反)再加 1 可得到该数的绝对值)。